

Lecture 3.

Programming Languages.

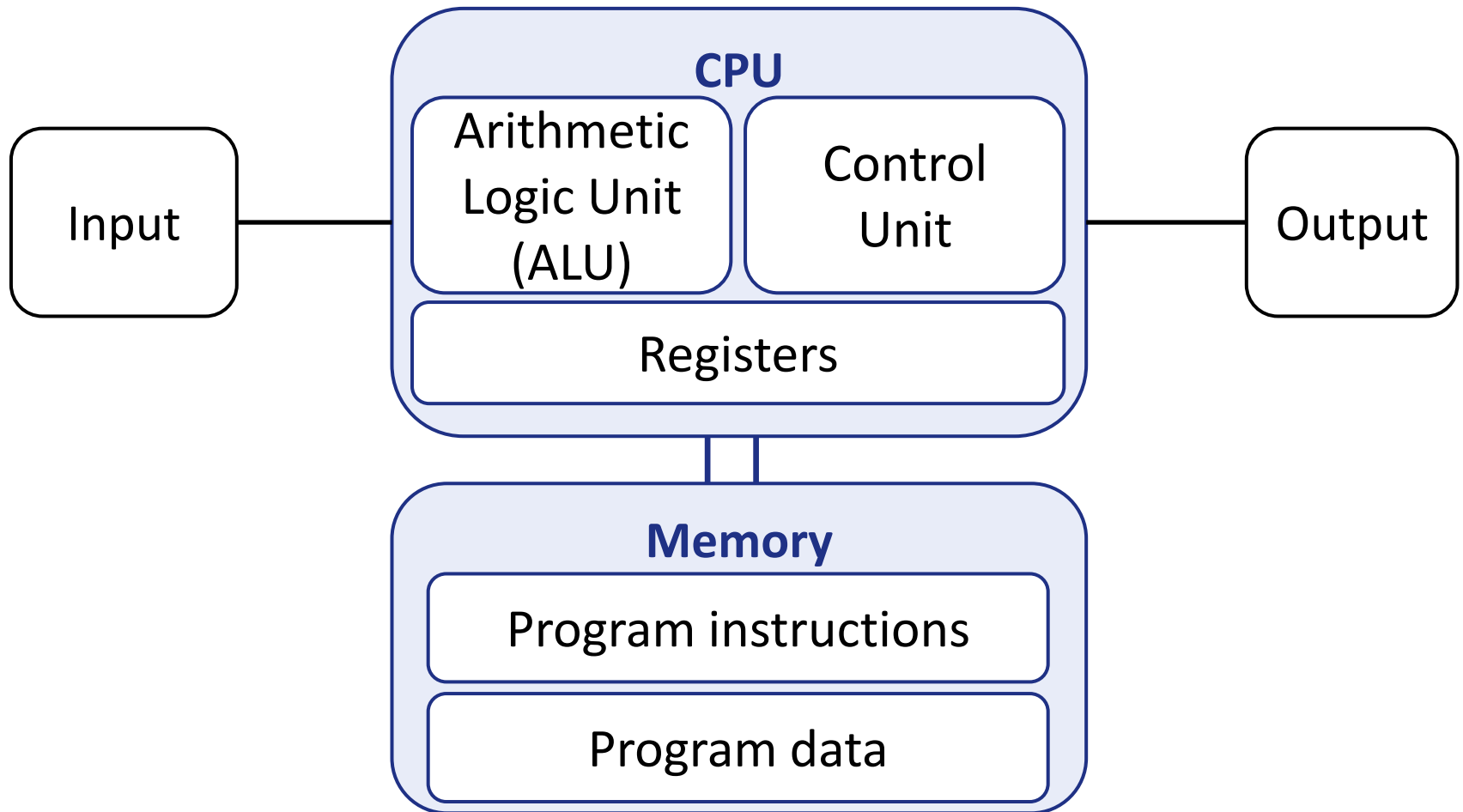
Software Development & Python
Ivan Solomatin
MIPT

Outline

- Why high-level programming languages exist
- Tokens and syntax trees
- Compilers and interpreters
- Bytecode and virtual machines
- CPython
- JIT compilation
- Typing, performance and abstraction costs

Recap

The CPU executes instructions

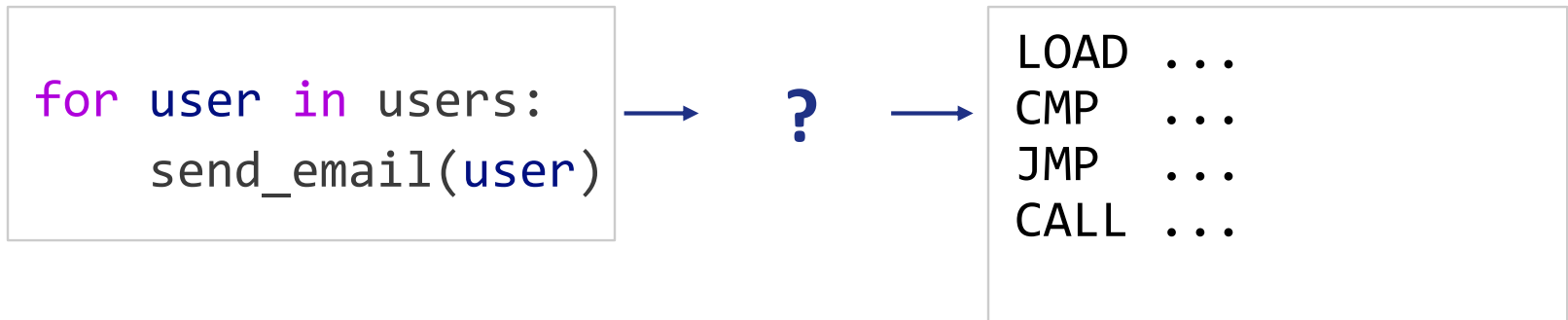


Fetch → Decode → Execute

But software developers do not normally write CPU instructions.

Recap

There is a gap between source code and the CPU



This lecture fills in the “?”

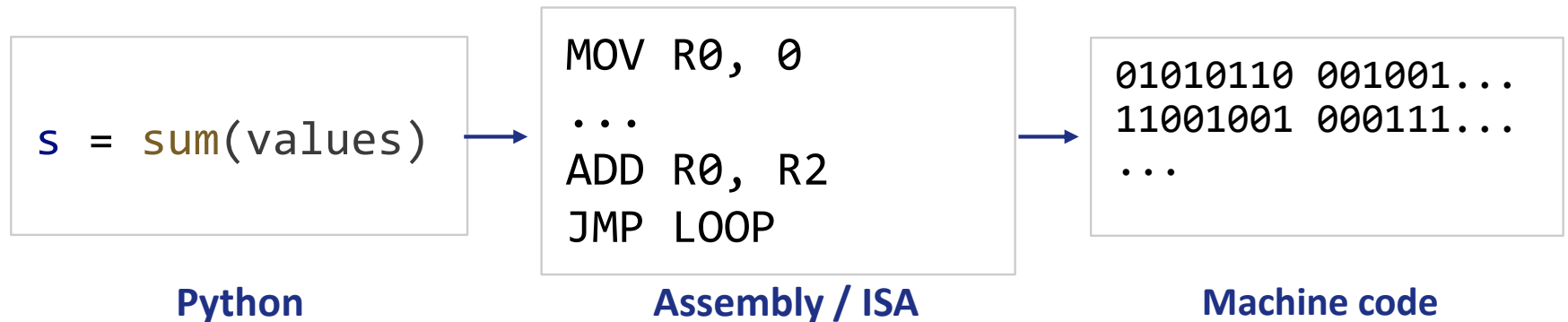
Why programming languages?

Why programming languages?

Main scheme

The CPU already has an instruction set — why invent another language?

The same computation at different abstraction levels:



Higher-level languages let us describe intent instead of machine details.

Why programming languages?

What do high-level languages buy us?

Abstraction

hide low-level details

Readability

express ideas closer to the problem

Portability

target different machines

Safety

prevent or detect classes of errors

Libraries

reuse enormous amounts of existing software

Productivity

optimize developer time, not only CPU time

Lexing and parsing

Lexing and parsing

Source code starts as text:

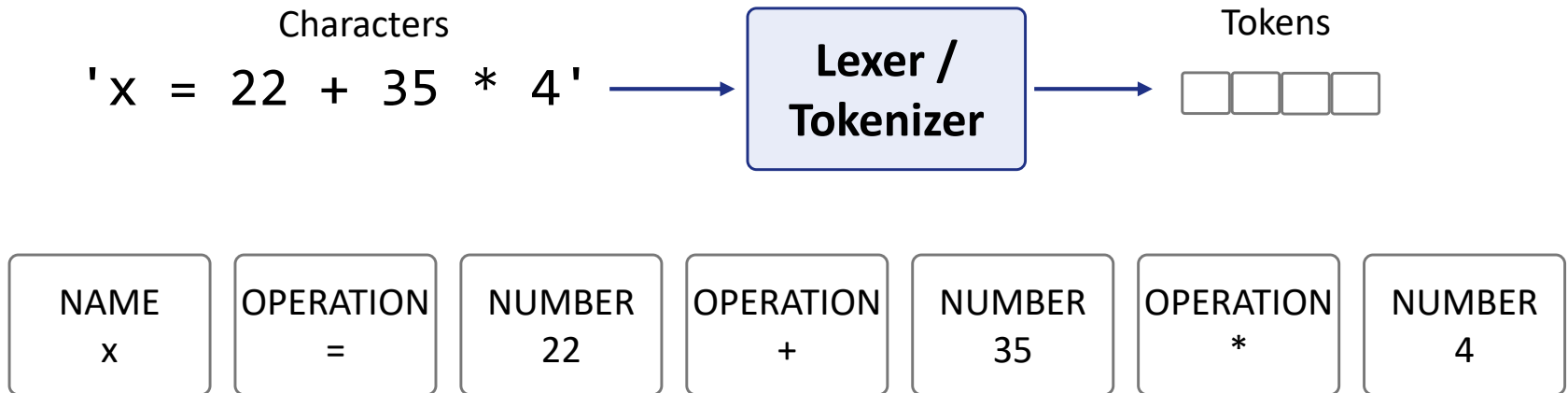
`x = 22 + 35 * 4`

To the language implementation this is initially just a sequence of characters.

`['x', '=', '2', '2', '+', '3', '5', '*', '4']`

Lexing and parsing

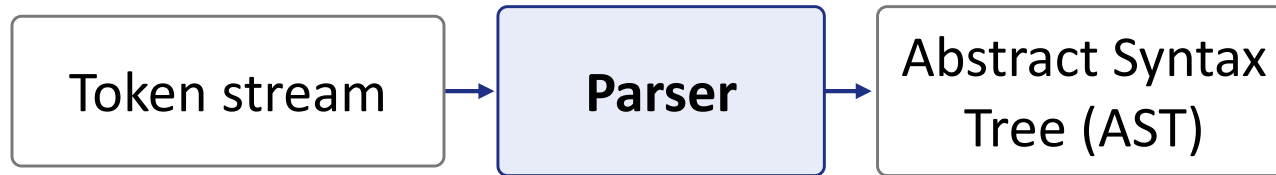
Step 1: Lexical analysis (tokenization)



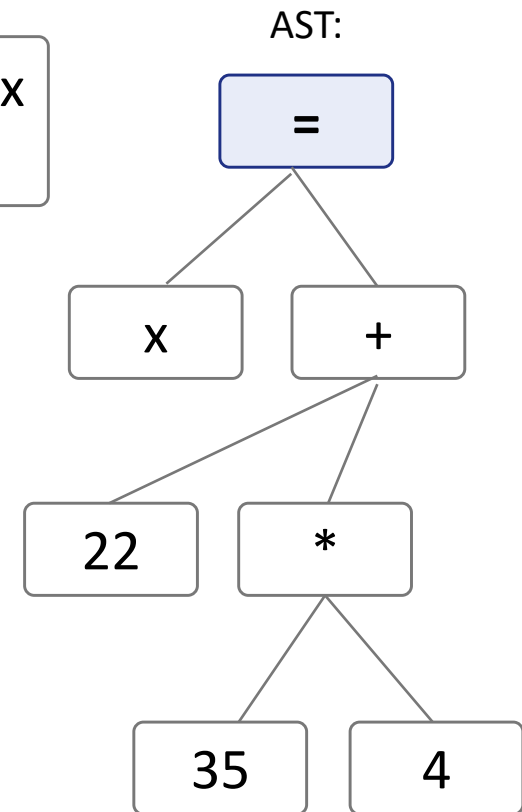
Tokens classify pieces of the source code.

Lexing and parsing

Step 2: Syntax analysis (parsing)



The parser does not “execute” the expression.
It discovers its syntactic structure.



Lexing and parsing

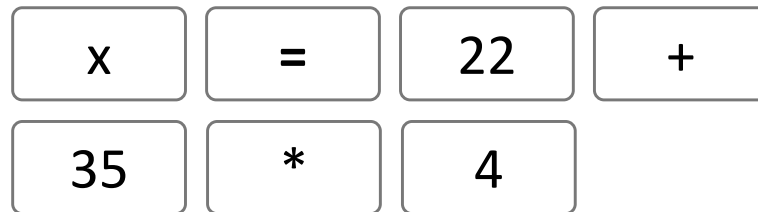
Abstract Syntax Tree (AST)

Characters

x = 22 + 35 * 4

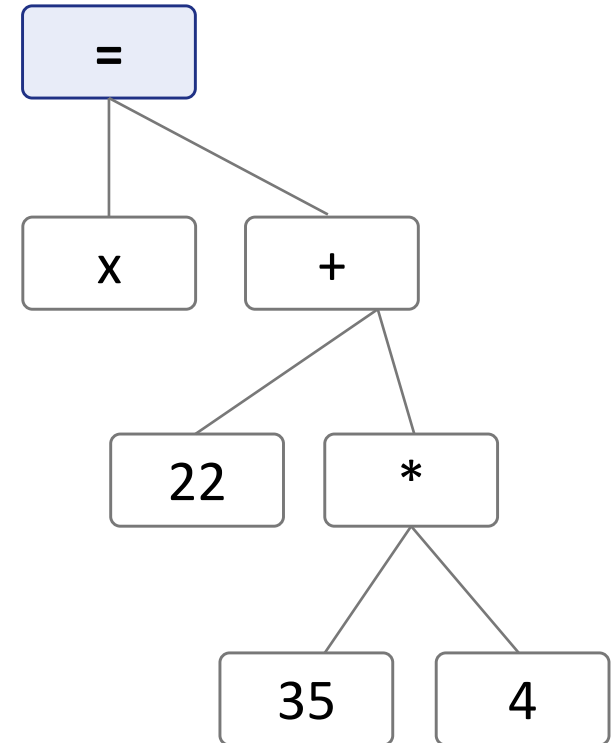
Tokenizer/Lexer

Tokens



Parser

AST



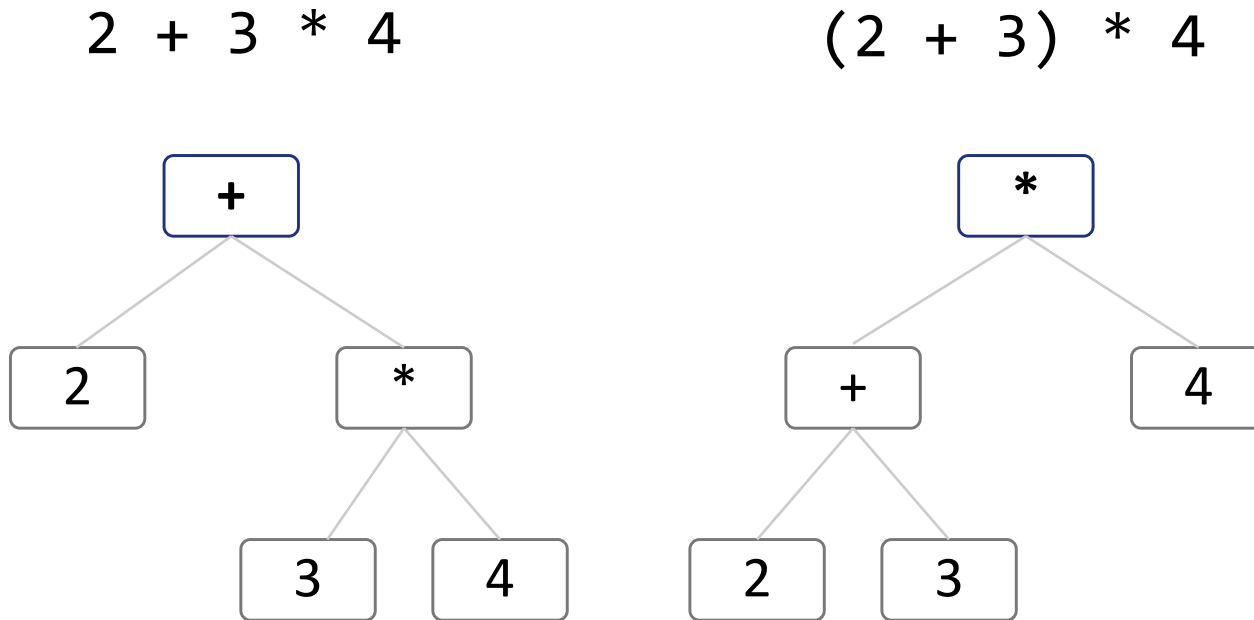
AST represents the **syntactic structure of a program**, but in a simplified tree form that omits details that are not important for meaning.

Simply saying, it represents the structure of a program: its operations, constructs, and their operands or arguments.

Lexing and parsing

Abstract Syntax Tree (AST)

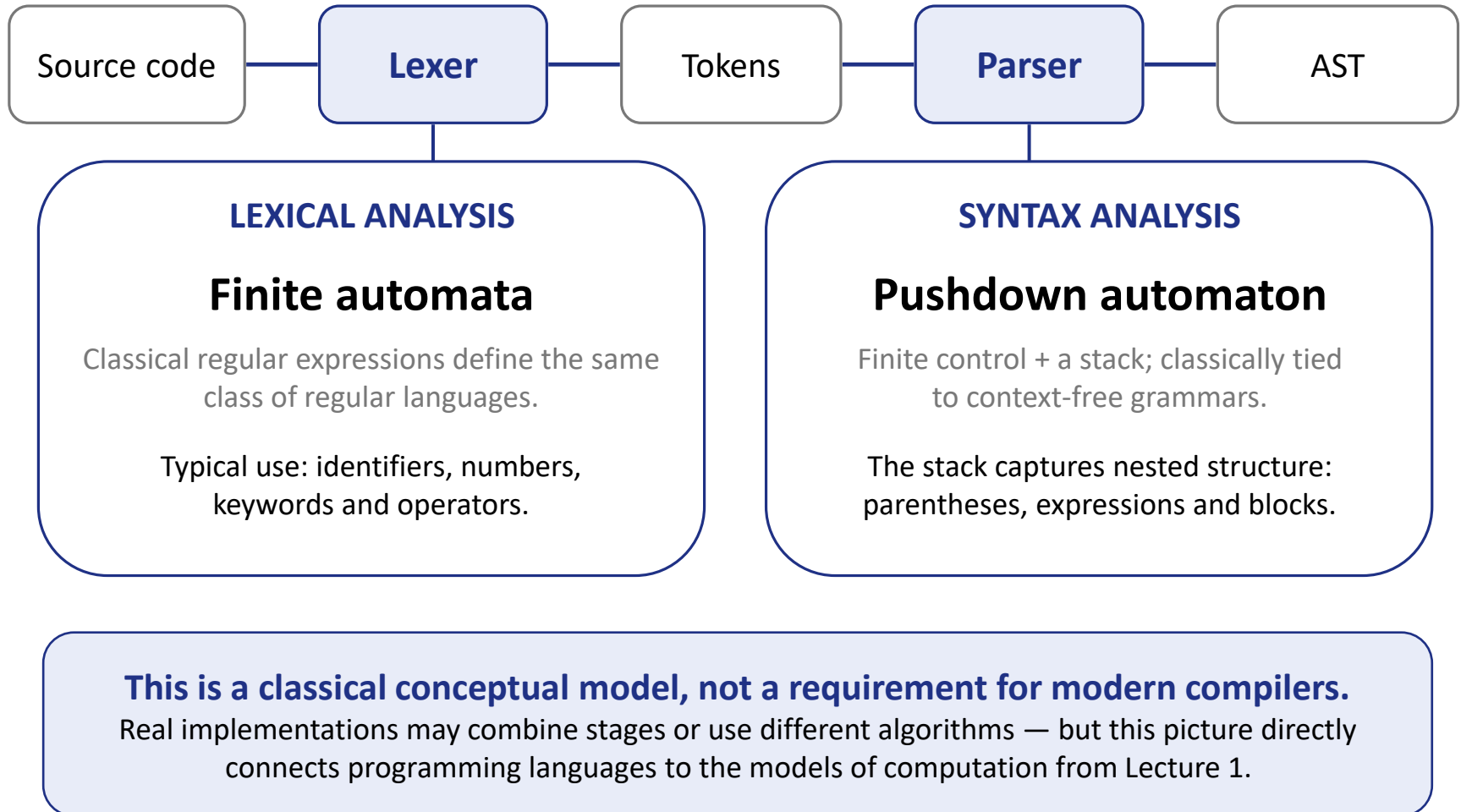
Precedence is encoded in the tree:



Different source structure → different AST → different result

Lexing and parsing

Connection to models of computation

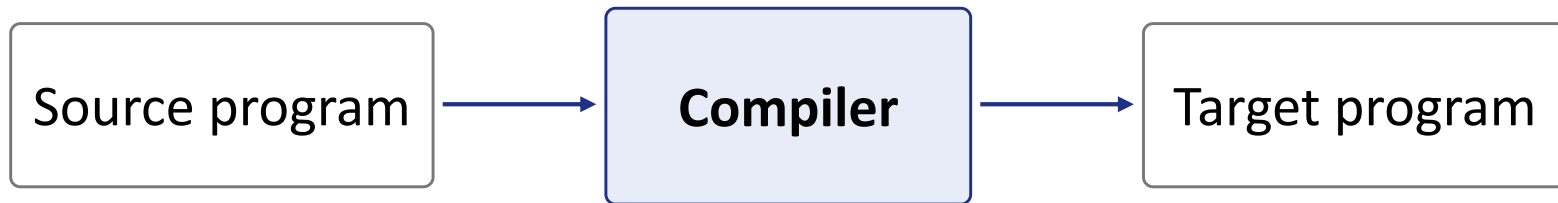


Compilers

Compilers

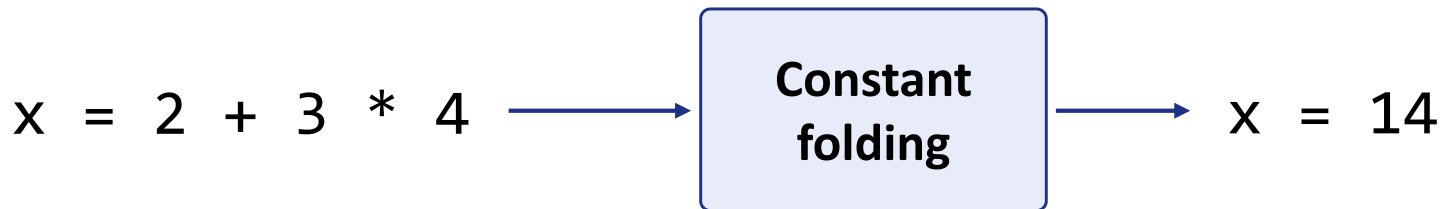
What they do?

Compilers transform one representation of a program into another.
I.e. translate programs from one language to another.



The target may be machine code, bytecode, another language, or an intermediate representation.

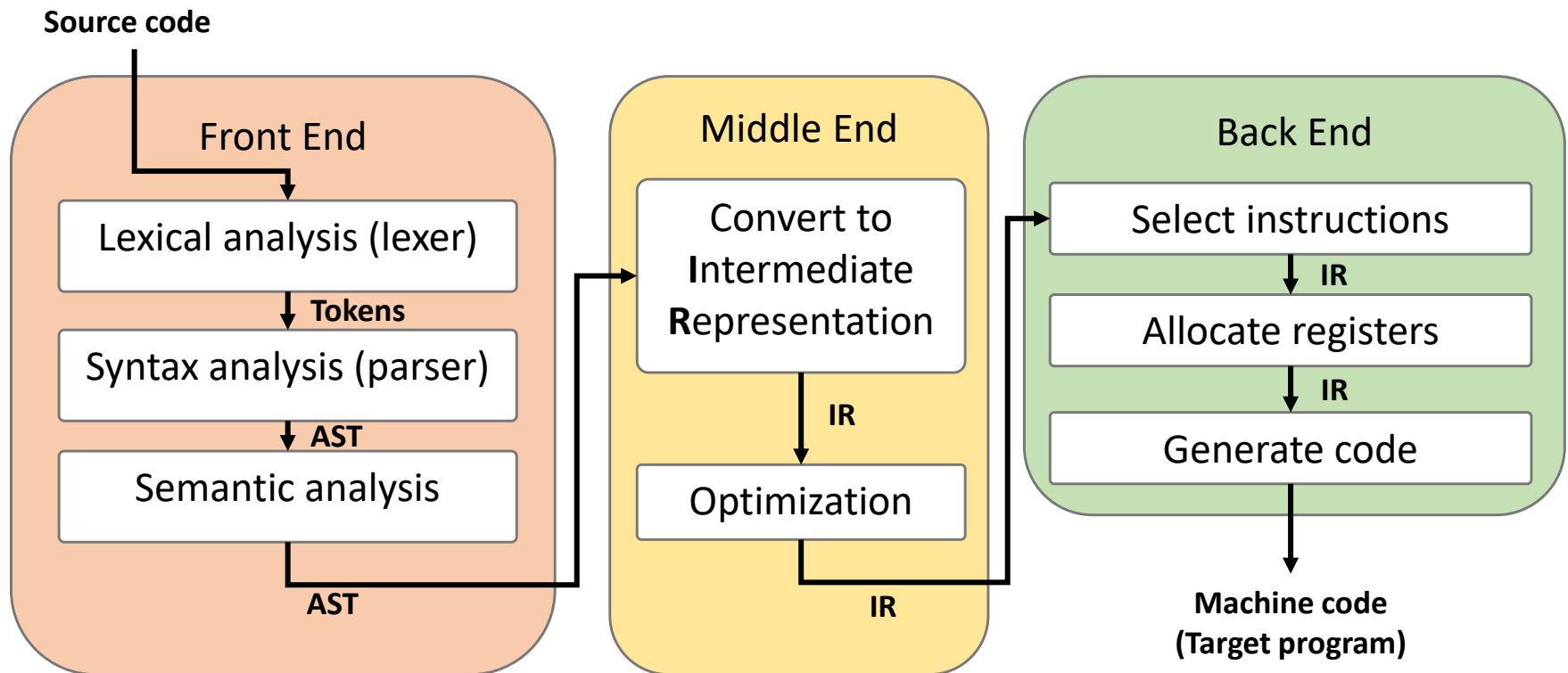
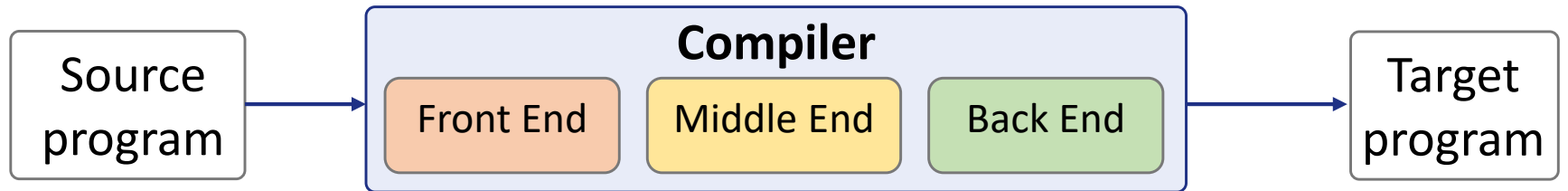
Compilation can also transform the program (optimizations):



Optimization should preserve observable behavior.

Compilers

Stages



Compilers

Common types

WHEN compilation happens

Ahead-of-Time (AOT)

source → native code

Compiled before execution. Typical examples:
C, C++, Rust.

Just-in-Time (JIT)

IR / bytecode → native code

Compiled during execution, often only for frequently executed (“hot”) code.

WHAT / WHERE it compiles to

Native compiler

Emits native machine code

Example: gcc, g++

Cross-compiler

host platform ≠ target platform

Runs on one platform but generates code for another. Example: x86 host → ARM target.

Bytecode compiler

source → VM bytecode

Targets a virtual machine. Examples: Java, CPython.

Source-to-source / transpiler

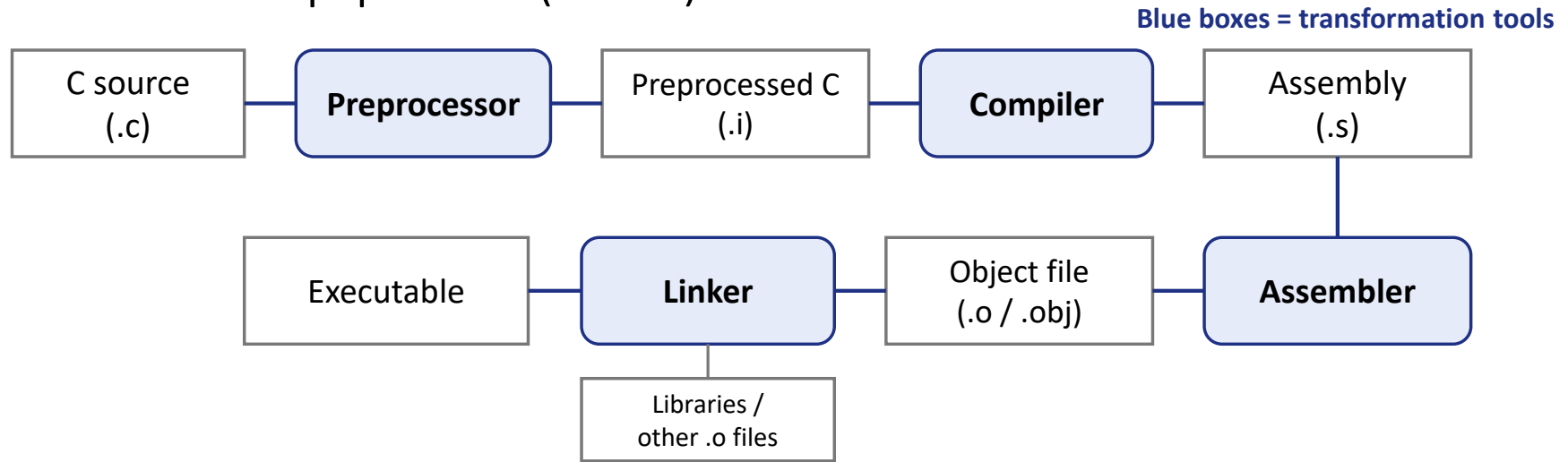
source → source

Produces another source language. Example:
TypeScript → JavaScript.

These categories overlap: one compiler may be AOT + cross-compiler, or part of a bytecode + JIT pipeline.

Compilers

Traditional pipeline (for C)



What each stage means

Preprocessor — handles `#include`, `#define` and conditional-compilation directives.

Compiler — translates the preprocessed source into assembly.

Assembly — a human-readable textual representation of CPU instructions.

Assembler — converts assembly into machine code stored in an object file.

Object file — machine code plus symbols and relocation metadata; usually not runnable by itself.

Linker — combines object files and libraries, resolves external symbols and addresses.

Executable — the final binary image that the operating system can load and start.

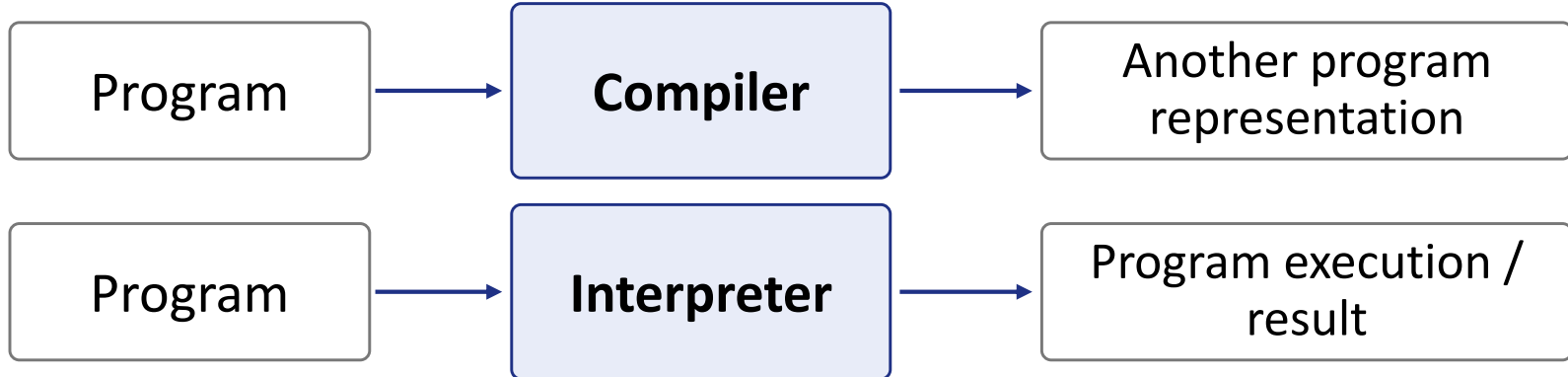
In practice, gcc/clang can drive all of these stages with a single command.

Interpreters

Interpreters

Compiler vs interpreter: two different jobs

A **compiler translates**, then we may run the program;
an **interpreter translates by running**.



A language itself is not inherently “compiled” or “interpreted”.

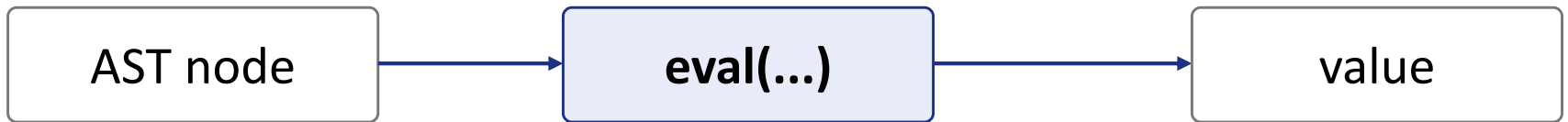
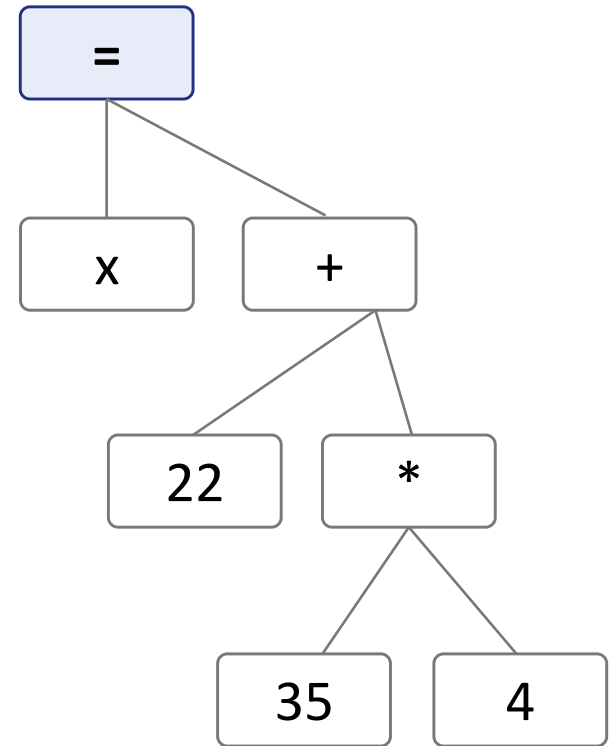
Many languages have both implementations.

When we say "C is a compiled language", this means "C is typically compiled".

Interpreters

A tiny interpreter can be surprisingly small

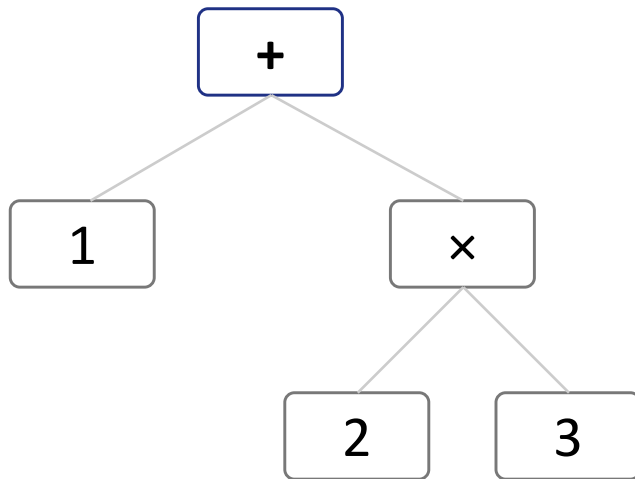
```
def eval(node):  
    match node:  
        case Number(v):  
            return v  
        case Add(a, b):  
            return eval(a) + eval(b)  
        case Mul(a, b):  
            return eval(a) * eval(b)
```



Interpreters

Example

Interpreting $1 + 2 \times 3$

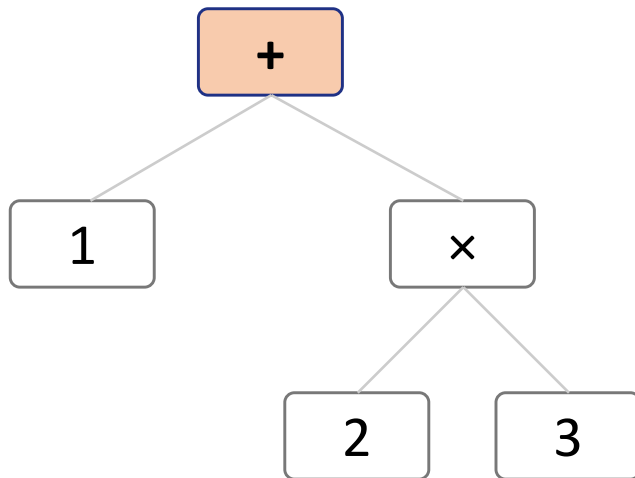


**The interpreter walks the structure
and performs its semantics.**

Interpreters

Example

Interpreting $1 + 2 \times 3$



$\text{eval}(+) \rightarrow \text{eval}(1)$

$\text{eval}(1) \rightarrow 1$

$\text{eval}(+) \rightarrow \text{eval}(x)$

$\text{eval}(x) \rightarrow \text{eval}(2) \rightarrow 2$

$\text{eval}(x) \rightarrow \text{eval}(3) \rightarrow 3$

$\text{eval}(x) \rightarrow 2 \times 3 \rightarrow 6$

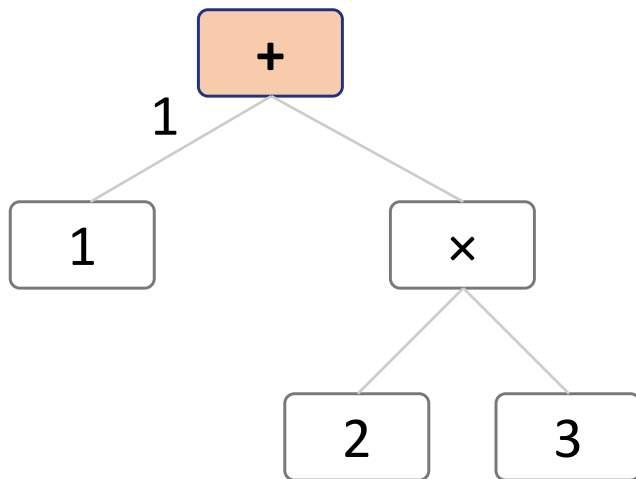
$\text{eval}(+) \rightarrow 1 + 6 \rightarrow 7$

**The interpreter walks the structure
and performs its semantics.**

Interpreters

Example

Interpreting $1 + 2 \times 3$



$\text{eval}(+) \rightarrow \text{eval}(1)$

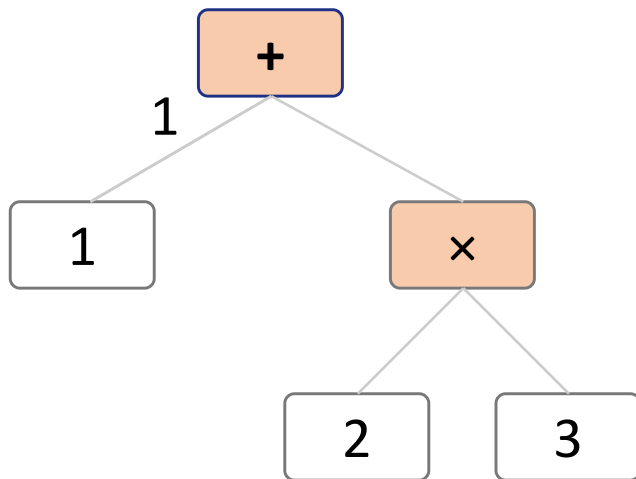
$\text{eval}(1) \rightarrow 1$

**The interpreter walks the structure
and performs its semantics.**

Interpreters

Example

Interpreting $1 + 2 \times 3$



$\text{eval}(+) \rightarrow \text{eval}(1)$

$\text{eval}(1) \rightarrow 1$

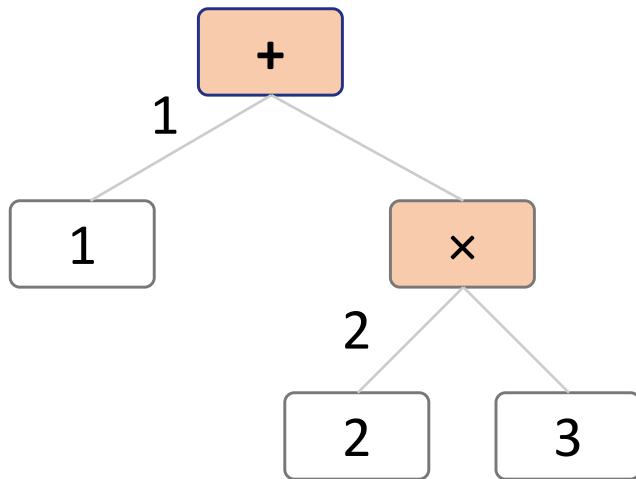
$\text{eval}(+) \rightarrow \text{eval}(\times)$

**The interpreter walks the structure
and performs its semantics.**

Interpreters

Example

Interpreting $1 + 2 \times 3$



$\text{eval}(+) \rightarrow \text{eval}(1)$

$\text{eval}(1) \rightarrow 1$

$\text{eval}(+) \rightarrow \text{eval}(\times)$

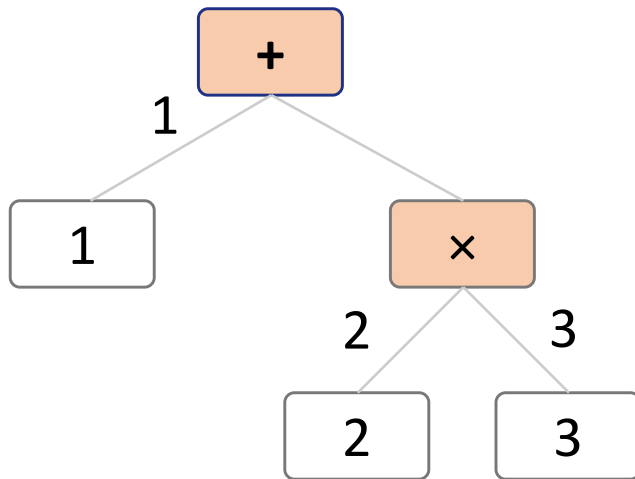
$\text{eval}(\times) \rightarrow \text{eval}(2) \rightarrow 2$

**The interpreter walks the structure
and performs its semantics.**

Interpreters

Example

Interpreting $1 + 2 \times 3$



$\text{eval}(+) \rightarrow \text{eval}(1)$

$\text{eval}(1) \rightarrow 1$

$\text{eval}(+) \rightarrow \text{eval}(\times)$

$\text{eval}(\times) \rightarrow \text{eval}(2) \rightarrow 2$

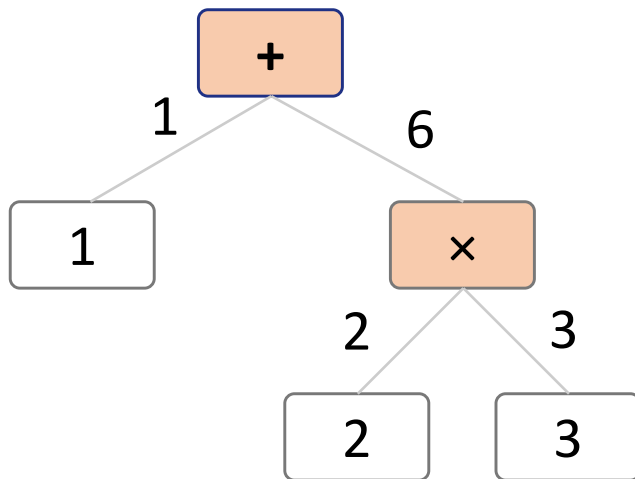
$\text{eval}(\times) \rightarrow \text{eval}(3) \rightarrow 3$

**The interpreter walks the structure
and performs its semantics.**

Interpreters

Example

Interpreting $1 + 2 \times 3$



$\text{eval}(+) \rightarrow \text{eval}(1)$

$\text{eval}(1) \rightarrow 1$

$\text{eval}(+) \rightarrow \text{eval}(\times)$

$\text{eval}(\times) \rightarrow \text{eval}(2) \rightarrow 2$

$\text{eval}(\times) \rightarrow \text{eval}(3) \rightarrow 3$

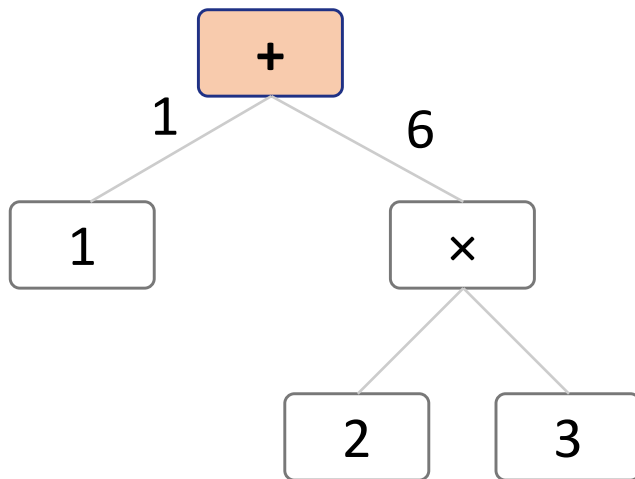
$\text{eval}(\times) \rightarrow 2 \times 3 \rightarrow 6$

**The interpreter walks the structure
and performs its semantics.**

Interpreters

Example

Interpreting $1 + 2 \times 3$



$\text{eval}(+) \rightarrow \text{eval}(1)$

$\text{eval}(1) \rightarrow 1$

$\text{eval}(+) \rightarrow \text{eval}(x)$

$\text{eval}(x) \rightarrow \text{eval}(2) \rightarrow 2$

$\text{eval}(x) \rightarrow \text{eval}(3) \rightarrow 3$

$\text{eval}(x) \rightarrow 2 \times 3 \rightarrow 6$

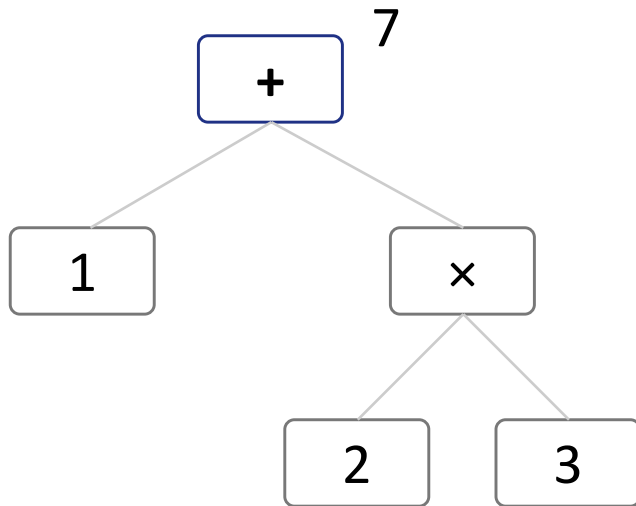
$\text{eval}(+) \rightarrow 1 + 6 \rightarrow 7$

**The interpreter walks the structure
and performs its semantics.**

Interpreters

Example

Interpreting $1 + 2 \times 3$



$\text{eval}(+) \rightarrow \text{eval}(1)$

$\text{eval}(1) \rightarrow 1$

$\text{eval}(+) \rightarrow \text{eval}(x)$

$\text{eval}(x) \rightarrow \text{eval}(2) \rightarrow 2$

$\text{eval}(x) \rightarrow \text{eval}(3) \rightarrow 3$

$\text{eval}(x) \rightarrow 2 \times 3 \rightarrow 6$

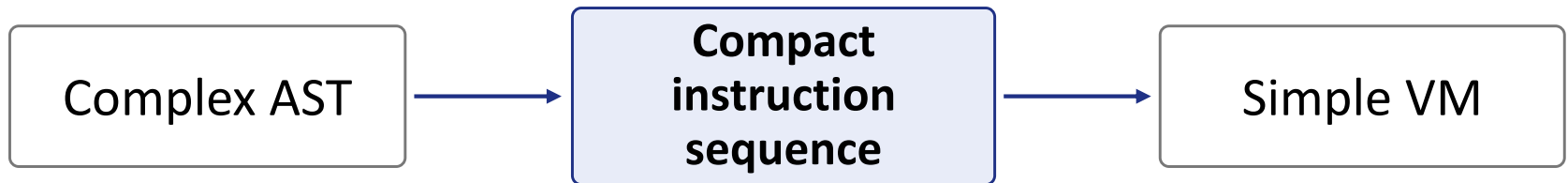
$\text{eval}(+) \rightarrow 1 + 6 \rightarrow 7$

**The interpreter walks the structure
and performs its semantics.**

Bytecode & virtual machines

Bytecode & virtual machines

Bytecode is a practical middle ground between AST interpretation and native code.



- easier to execute repeatedly
- portable across physical CPUs
- simpler than native machine code
- good target for compilers and optimizers

Bytecode & virtual machines

Example

$2 * 3 + 1$

```
PUSH 1  
PUSH 2  
PUSH 3  
MUL  
ADD  
HALT
```

Stack



The VM has a tiny instruction set and an evaluation stack.

Bytecode & virtual machines

Example

$2 * 3 + 1$

Current instruction: **PUSH 1**

PUSH 1

PUSH 2

PUSH 3

MUL

ADD

HALT

Push value 1

Stack

1

Bytecode & virtual machines

Example

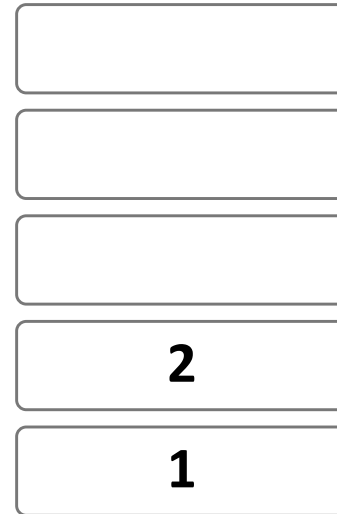
$2 * 3 + 1$

Current instruction: **PUSH 2**

```
PUSH 1  
PUSH 2  
PUSH 3  
MUL  
ADD  
HALT
```

Push value 2

Stack



Bytecode & virtual machines

Example

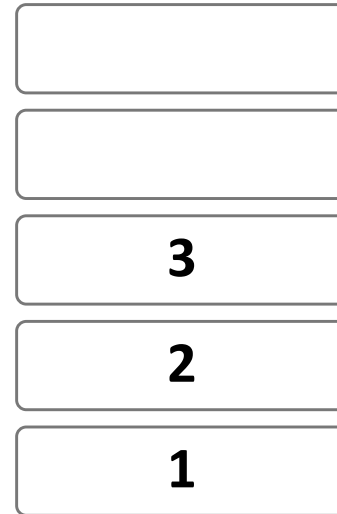
$2 * 3 + 1$

Current instruction: **PUSH 3**

```
PUSH 1  
PUSH 2  
PUSH 3  
MUL  
ADD  
HALT
```

Push value 3

Stack



Bytecode & virtual machines

Example

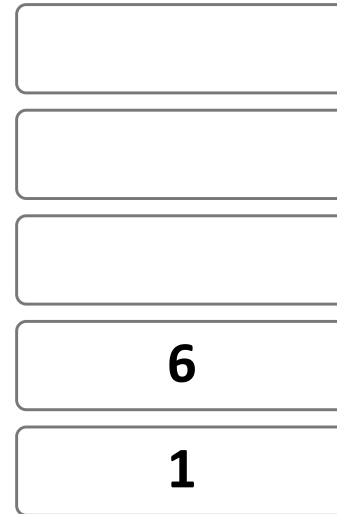
$2 * 3 + 1$

Current instruction: **MUL**

```
PUSH 1  
PUSH 2  
PUSH 3  
MUL  
ADD  
HALT
```

**Pop 3 and 2
Push 2+3=6**

Stack



Bytecode & virtual machines

Example

$2 * 3 + 1$

Current instruction: ADD

PUSH 1
PUSH 2
PUSH 3
MUL
ADD
HALT

**Pop 6 and 1
Push 7**

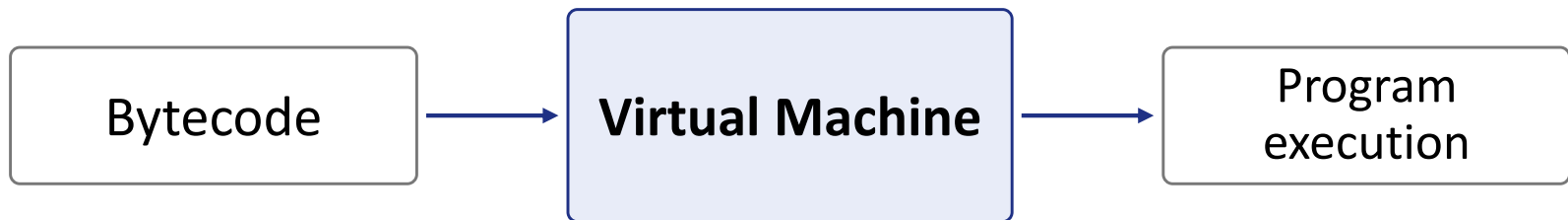
Stack



Bytecode & virtual machines

Virtual machine

Virtual machine = an abstract processor implemented in software



The VM defines its own instructions, state, stack/registers and execution rules.

This should feel familiar: TinyCPU was also a virtual machine.

CPython

CPython

What it is?

CPython is the standard and most widely used implementation of the Python programming language, written primarily in C.

CPython is a Python interpreter that compiles Python source code into Python bytecode and executes that bytecode using the CPython virtual machine.

Simplified pipeline:



CPython

Bytecode

```
def add(a, b):  
    return a + b  
  
import dis  
dis.dis(add)
```

Illustrative bytecode — exact details vary between CPython versions.

```
LOAD_FAST    a  
LOAD_FAST    b  
BINARY_OP    +  
RETURN_VALUE
```

Python bytecode is not x86 / ARM machine code.

A bytecode instruction is an instruction for the Python VM

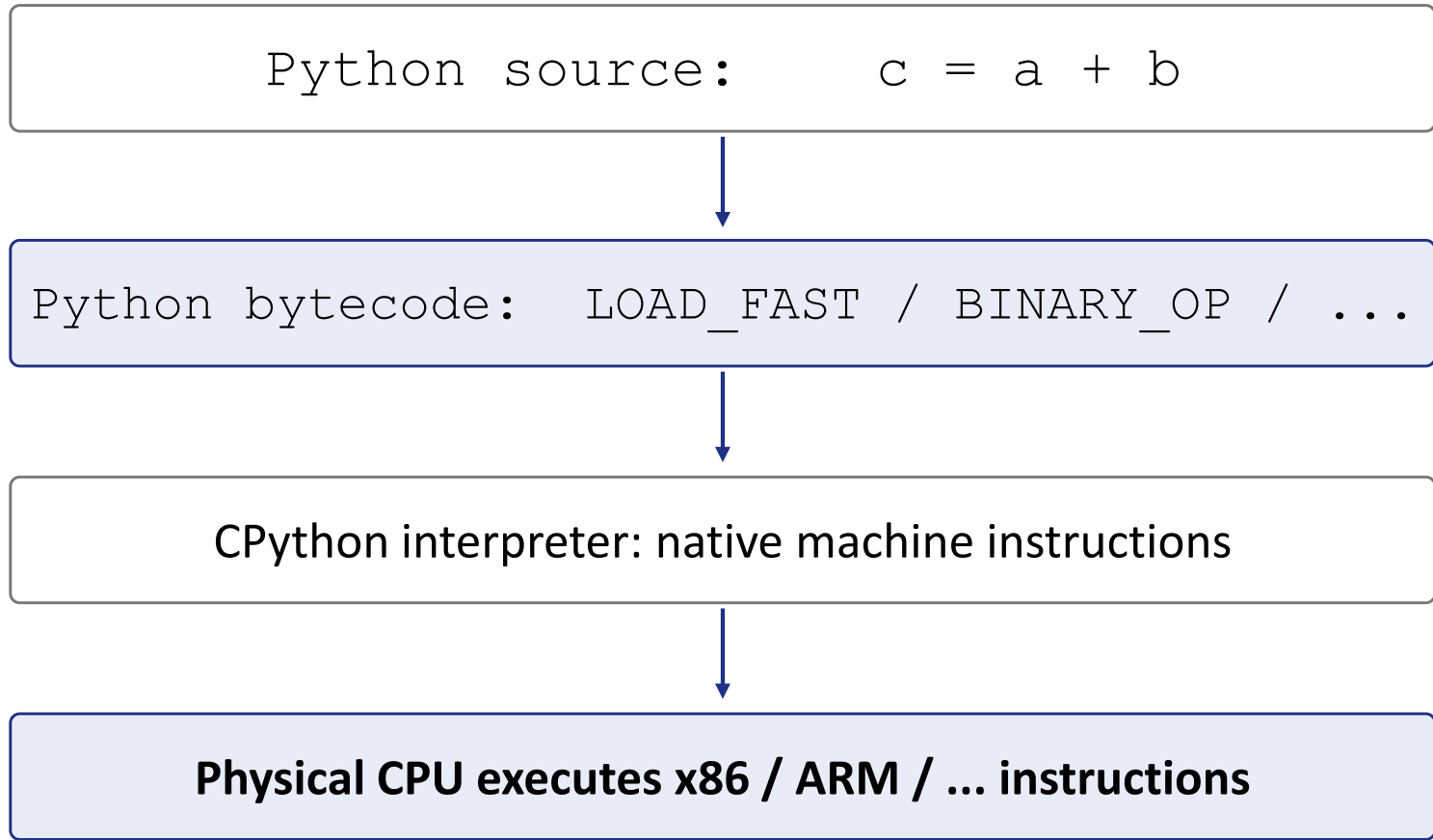


The CPython interpreter itself is a native program running on the real CPU.

So there are multiple instruction layers.

CPython

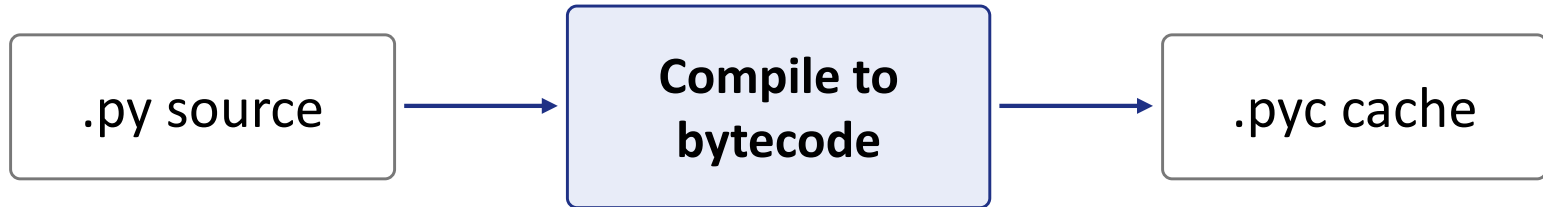
Instruction layers



CPython

Caching

Compiled Python bytecode can be cached



Caching avoids repeating some compilation work on later imports.

It does not turn Python bytecode into native machine code.

CPython

Language $\neq$ implementation $\neq$ runtime

Python

**Language specification
/ semantics**

CPython

**One implementation
of Python**

Runtime

**Services available
while code runs**

Other implementations can make different execution choices while implementing the same language.

Runtime provides for example:

object representation

memory management

function calls

exceptions

**interaction with the
operating system**

threads / concurrency
support

A high-level language shifts work from the programmer to the runtime.

JIT (Just-In-Time) compilation

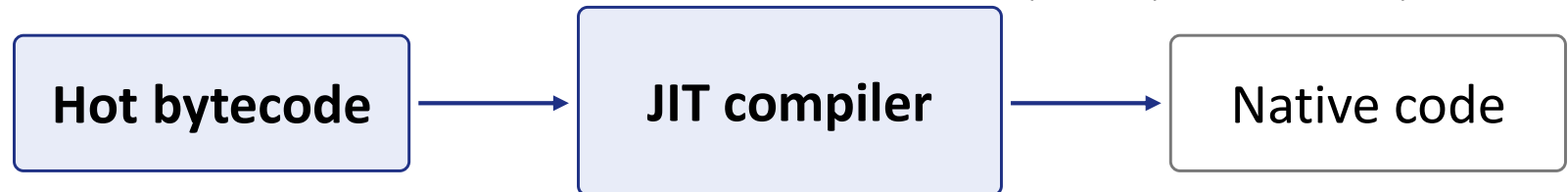
JIT (Just-In-Time) compilation

Idea: Interpret first, compile hot code when it becomes worthwhile.

1. First execute normally

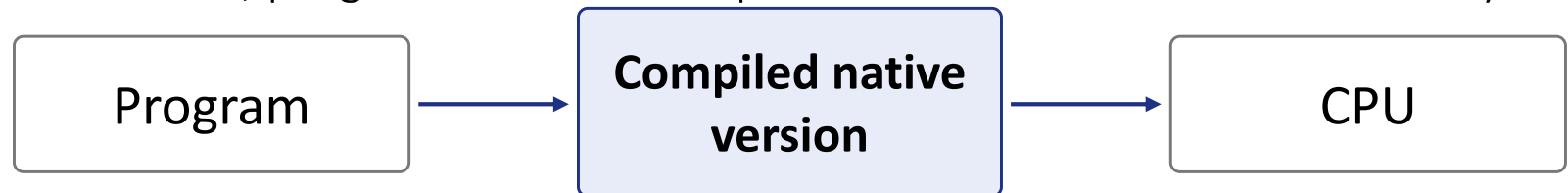


2. The runtime observes which code executes frequently. Then compiles hot one.



Compilation happens only when the expected payoff justifies the cost.

3. After that, program refers to compiled code which is executed directly on CPU



JIT (Just-In-Time) compilation

“Compiled vs interpreted language” is an oversimplification

C

source → native code

CPython

source → bytecode → interpreter

PyPy

source → bytecode → interpreter → JIT → native code

Java

source → bytecode → JVM / JIT → native code

**JavaScript
engines**

source → internal representations → interpreter / JIT

Think in terms of execution pipelines, not labels.

Typing & abstraction costs.

Static/dynamic typing

Typing & abstraction costs.

Static/dynamic typing

Two different questions:

When are type constraints checked?

Static ↔ Dynamic

How is code executed?

AOT ↔ Interpreter ↔ JIT

Do not assume “compiled = static” or “interpreted = dynamic”.

Static/dynamic typing

C

```
int x = 10;  
x = "hello";  
// rejected
```

Static typing: more information is established before execution.

Python

```
x = 10  
x = "hello"  
# valid binding
```

Dynamic typing: values carry runtime type information and operations are resolved while running.

Typing & abstraction costs.

Static/dynamic typing

Why can a Python operation cost more than a machine ADD?

`c = a + b`

What objects are a and b?

Which addition operation applies?

Can user-defined behavior run?

How is the result object represented?

How are references / memory managed?

Dynamic flexibility moves work into runtime.

Typing & abstraction costs.

Tradeoffs

C:

More information known
ahead of time

Less runtime machinery

More manual responsibility

Python:

More dynamic behavior

More runtime services

Higher developer-level
abstraction

Neither trade-off is universally “better”.

Typing & abstraction costs.

Tradeoffs

High-level Python can still drive low-level optimized code



NumPy, PyTorch and many scientific libraries use this pattern.

High-level syntax does not imply that every arithmetic operation is executed by Python bytecode.

Conclusion

Conclusion

“Virtual machine” is used in two different senses

Language VM

**Python VM
JVM
.NET CLR**

**Executes a language-level
instruction model**

System VM

**VirtualBox
VMware
KVM**

**Virtualizes an entire computer
/ OS environment**

Same phrase, very different abstraction level.

Conclusion

What should you remember?

- Source code must first be transformed into structured representations such as tokens and ASTs.
- A compiler transforms one program representation into another; an interpreter executes a representation.
- Bytecode defines instructions for a virtual machine, not for the physical CPU.
- CPython compiles Python source to bytecode and executes it in the CPython VM.
- Modern runtimes can combine interpretation and JIT compilation.
- Typing strategy and execution strategy are separate design dimensions.
- High-level languages balance runtime performance against abstraction and developer productivity.

Conclusion

We now understand the execution pipeline



Thank you for watching!